

CSCI322

Systems Administration

School of Computing and Information Technology
University of Wollongong

Courseware developed with input from Daniel Saffioti and William Tibben



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

What is an operating system?

- The executive manager that manages all of the hardware and all of the software.
 - Memory manager
 - Process manager
 - Device manager
 - File manager
 - Network manager
 - User command interface



The UNIX operating system

- A portable operating system
- A multi-tasking operating system
- A multi-user computer operating system in a time-sharing configuration
- Provides a standard command-line interface



Friction between UNIX and Linux

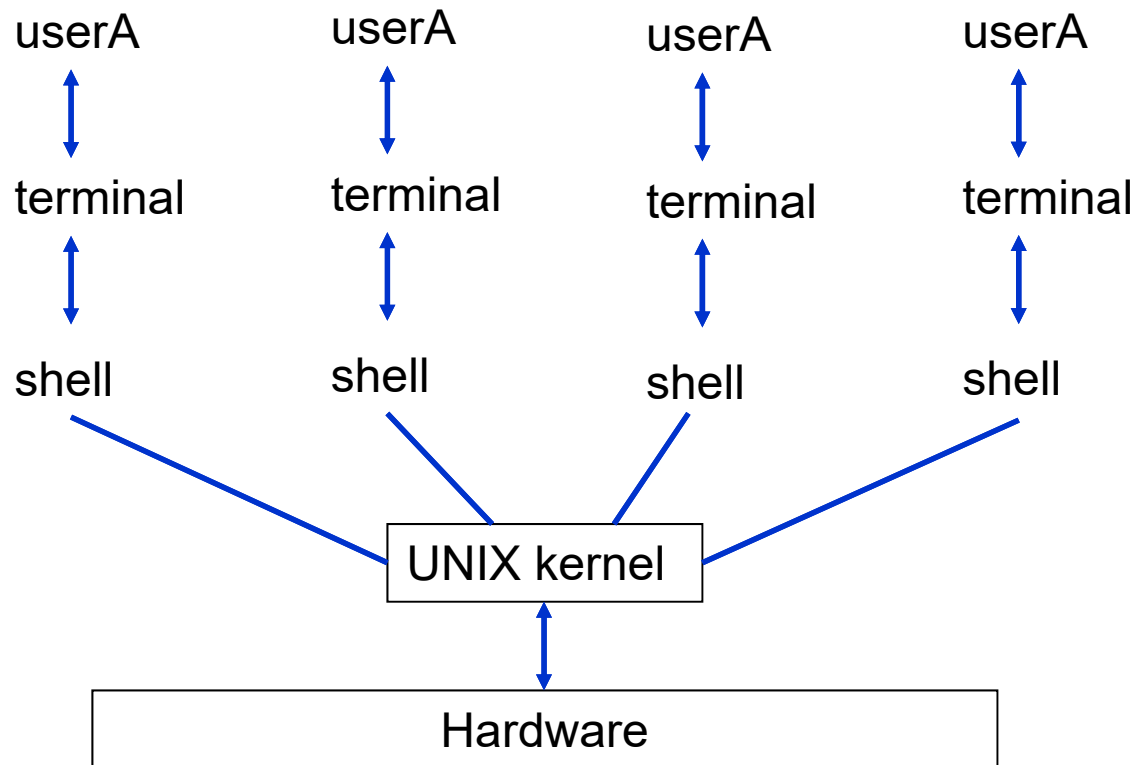
- Linux was developed in 1991 in Finland
- Linux is a reimplementation and elaboration of the UNIX kernel.
- Linux conforms to the POSIX standard.
- Linux is free, open source, and cooperatively developed.
- Many distributions: Debian, Linux Mint, Red Hat Enterprise, SUSE Linux Enterprise, Ubuntu, etc.
- There are other UNIX flavors like AIX, FreeBSD, HP-UX, Solaris, Unixware



Accessing a UNIX system

- The kernel is a core component that is loaded when you first turn your computer on.
- The kernel loads all other components and serves to centrally control the activities of the computer.
- A user interacts with the kernel through a channel called terminal.
- After logging into a command-line terminal, the user receives a user interface called a shell.





Common UNIX shells

- Bourne shell (sh)
- POSIX shell (sh)
- Korn shell (ksh)
- C shell (csh)
- BASH shell (bash)

- When a user logs into a graphical terminal
 - X Windows is loaded to provide graphical functionality
 - A desktop environment is loaded to standardise the look and feel of X Windows



Basic shell commands for a command-line terminal

- Shell prompt
 - # for the root user
 - \$ for all other users
- Commands indicate the name of the program to execute and are case sensitive
- Options are specific letters that start with a dash (-) and appear after the command name to alter the way the command works.
- Arguments also appear after the command name, yet do not start with a dash. They specify the parameters that the command works upon.





Some common UNIX command

Command	Description
ls	List your files
more filename	Show the first part of a file
emacs filename	Create and edit a file
vi filename	Create and edit a file
mv fn1 fn2	Move a file
cp fn1 fn2	Copy a file
rm filename	Remove a file
diff fn1 fn2	Compare files, and show where they differ
wc filename	How many lines, words, and characters
chmod	Change the permission of a file



Some common UNIX command

Command	Description
gzip filename	Compress a file
gunzip filename	Uncompress a file
gzcat filename	Look at a gzipped file without having to gunzip it
cat filename	Look at the content of the file
lpr filename	Print
mkdir dirname	Make a new directory
cd dirname	Change the directory
pwd	Where you are
grep string fn(s)	Look for the string in the files.



Some common UNIX command

Command	Description
w	Who's logged in and what they are doing
who	Who's logged in and where they are coming from
finger	Display information on system users
last	Give a list of everyone's logins
id	Display the user id and group id
uname -a	Display all information about the system
date	Current date and time
cal	Display the calendar of the current month
exit	Exit the current shell



Some common UNIX command

Command	Description
whoami	Return your username
passwd	Change your password
ps	Display current processes
kill PID	Kill the process with the id you have
quota -v	Show what your disk quota is
echo	Print text to the terminal screen
mail	Send file by email
ssh	Open a secure connection
ftp	Upload or download file
!!	Repeat the previous command



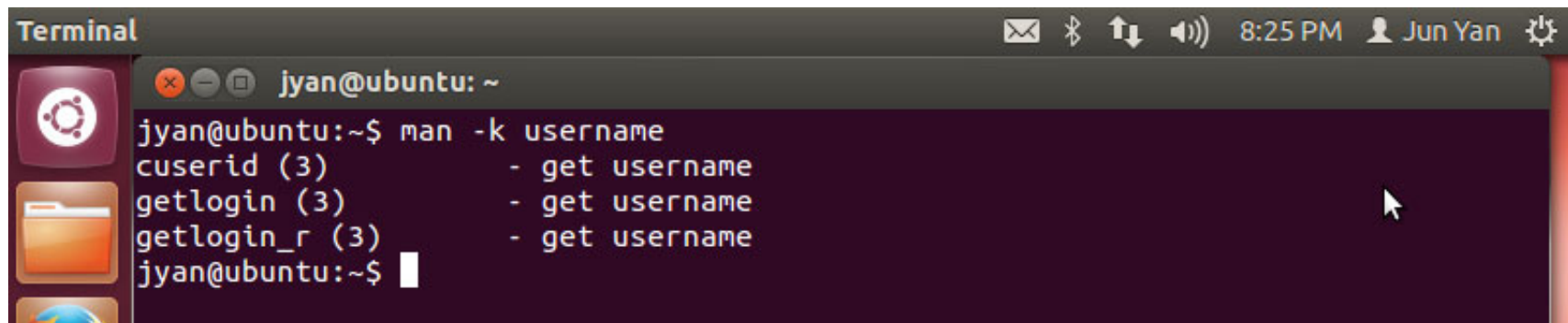
Obtain command help

- It is impractical to memorise the syntax and use of more than 1000 UNIX commands.
- Use the command *man* followed by the command name to retrieve the manual page of that command

`man date`

- Search the manual pages by keyword

`man -k keyword`

A screenshot of a Linux terminal window titled "Terminal". The window shows a user named "jyan" at a machine named "ubuntu" in the home directory. The user has entered the command "man -k username". The terminal output lists three manual pages: "cuserid (3)", "getlogin (3)", and "getlogin_r (3)", each followed by a description "- get username". The prompt "jyan@ubuntu:~\$" is visible at the bottom of the terminal.

```
Terminal
jyan@ubuntu: ~
jyan@ubuntu:~$ man -k username
cuserid (3)          - get username
getlogin (3)         - get username
getlogin_r (3)       - get username
jyan@ubuntu:~$
```

Manual page section number

Section number	Description
1 1M X1M	Commands that any user may execute Command that only root user may execute X Windows commands that only the root user may execute
2	System calls
3 3M 3F	Library routines and functions Mathematical functions Fortran functions
4	File formats
5	Miscellaneous
6	Games
7	Device drivers and interfaces



Shell metacharacters

Metacharacter	Description
\$	Shell variable
&	Background command execution
;	Command termination
< << > >>	I/O redirection
	Command piping
* ? []	Shell wildcards
' " \	Metacharacter quotes
`	Command substitution
() {}	Command grouping



Terminal

8:31 PM Jun Yan

```
jyan@ubuntu: ~  
jyan@ubuntu:~$ echo My Shell is $SHELL  
My Shell is /bin/bash  
jyan@ubuntu:~$ echo 'My Shell is $SHELL'  
My Shell is $SHELL  
jyan@ubuntu:~$ echo "My Shell is $SHELL"  
My Shell is /bin/bash  
jyan@ubuntu:~$ echo My Shell is \$SHELL  
My Shell is $SHELL  
jyan@ubuntu:~$ echo Today is `date`  
Today is Sun Feb 25 20:31:51 PST 2018  
jyan@ubuntu:~$
```

Command input and output

- Your shell can manipulate command input and output. Command input and output are represented by labels known as file descriptors.
 - Standard input (stdin), descriptor number 0, by default refers to the user input typed on the keyboard
 - Standard output (stdout), descriptor number 1, by default refers to the terminal screen
 - Standard error (stderr), descriptor number 2, by default refers to the terminal screen



Redirection

- Redirect standard output and standard error from the terminal screen to a file using the `>` shell metacharacter followed by the pathname of the file
- The shell assumes standard output in the absence of a numeric file descriptor
- You can also redirect both standard output and standard error to the same file using special notation (see next slide)
- You can redirect a file to the standard input of a command using the `<` metacharacter



```
Terminal
jyan@ubuntu: ~
jyan@ubuntu:~$ ls /etc/hosts /etc/h 1>goodoutput
ls: cannot access /etc/h: No such file or directory
jyan@ubuntu:~$ cat goodoutput
/etc/hosts
jyan@ubuntu:~$ ls /etc/hosts /etc/h 2>badoutput
/etc/hosts
jyan@ubuntu:~$ cat badoutput
ls: cannot access /etc/h: No such file or directory
jyan@ubuntu:~$ ls /etc/hosts /etc/h 1>goodoutput 2>&1
jyan@ubuntu:~$ cat goodoutput
ls: cannot access /etc/h: No such file or directory
/etc/hosts
jyan@ubuntu:~$ ls /etc/hosts /etc/h 1>>goodoutput 2>>&1
bash: syntax error near unexpected token `&'
jyan@ubuntu:~$ ls /etc/hosts /etc/h 1>>goodoutput 2>&1
jyan@ubuntu:~$ cat goodoutput
ls: cannot access /etc/h: No such file or directory
/etc/hosts
ls: cannot access /etc/h: No such file or directory
/etc/hosts
jyan@ubuntu:~$
```

```
Terminal
jyan@ubuntu: ~
jyan@ubuntu:~$ cat /etc/hosts
127.0.0.1    localhost
127.0.1.1    ubuntu

# The following lines are desirable for IPv6 capable hosts
::1        ip6-localhost ip6-loopback
fe00::0    ip6-localnet
ff00::0    ip6-mcastprefix
ff02::1    ip6-allnodes
ff02::2    ip6-allrouters
jyan@ubuntu:~$ tr l L </etc/hosts
127.0.0.1    Localhost
127.0.1.1    ubuntu

# The foLLowing Lines are desirable for IPv6 capable hosts
::1        ip6-Localhost ip6-Loopback
fe00::0    ip6-Localnet
ff00::0    ip6-mcastprefix
ff02::1    ip6-aLLnodes
ff02::2    ip6-aLLrouters
jyan@ubuntu:~$
```

Pipes

- To connect the stdout of one command to the stdin of another, use the | symbol, commonly known as a pipe
- You can use more than one | metacharacter on the command line to pipe information from one command to another
- Use the *tee* command to write to the standard output and a file
- You can also combine the redirection and pipe



Combine redirection and piping

- You can also combine redirection and piping together
 - Input direction must occur at the beginning of the pipe
 - Output direction must occur at the end of the pipe



Filter

- Any command that can take from standard input and give to standard output is called a filter.

Command	Description
cut	Prints selected portions of its input lines
sort sort -r	Sorts lines in a file (ASCII) Reverse sorts lines in a file (ASCII)
wc wc -l wc -w wc -c	Counts the number of lines, words, and characters in a file
pr pr -d	Formats a file for printing Formats a file double spaced
tr	Replaces characters in the text of a file
grep	Displays lines in a file that match a regular expression
nl	Numbers lines in a file

cut

- The *cut* command prints selected portions of its input lines. It's commonly used to extract delimited fields.
- The default delimiter is <Tab>
- You can change delimiters with the `-d` option
- The `-f` options specifies which fields to include in the output
- How to enumerate the system's users?



Sort

Option	Meaning
-b	Ignore leading whitespace
-f	Case insensitive sorting
-k	Specify the columns that form the sort key
-n	Compare fields as integer numbers
-r	Reverse sort order
-t	Set field separator (the default is whitespace)
-u	Output unique records only

How to reverse sort the /etc/group file by its group ID?



Exercises

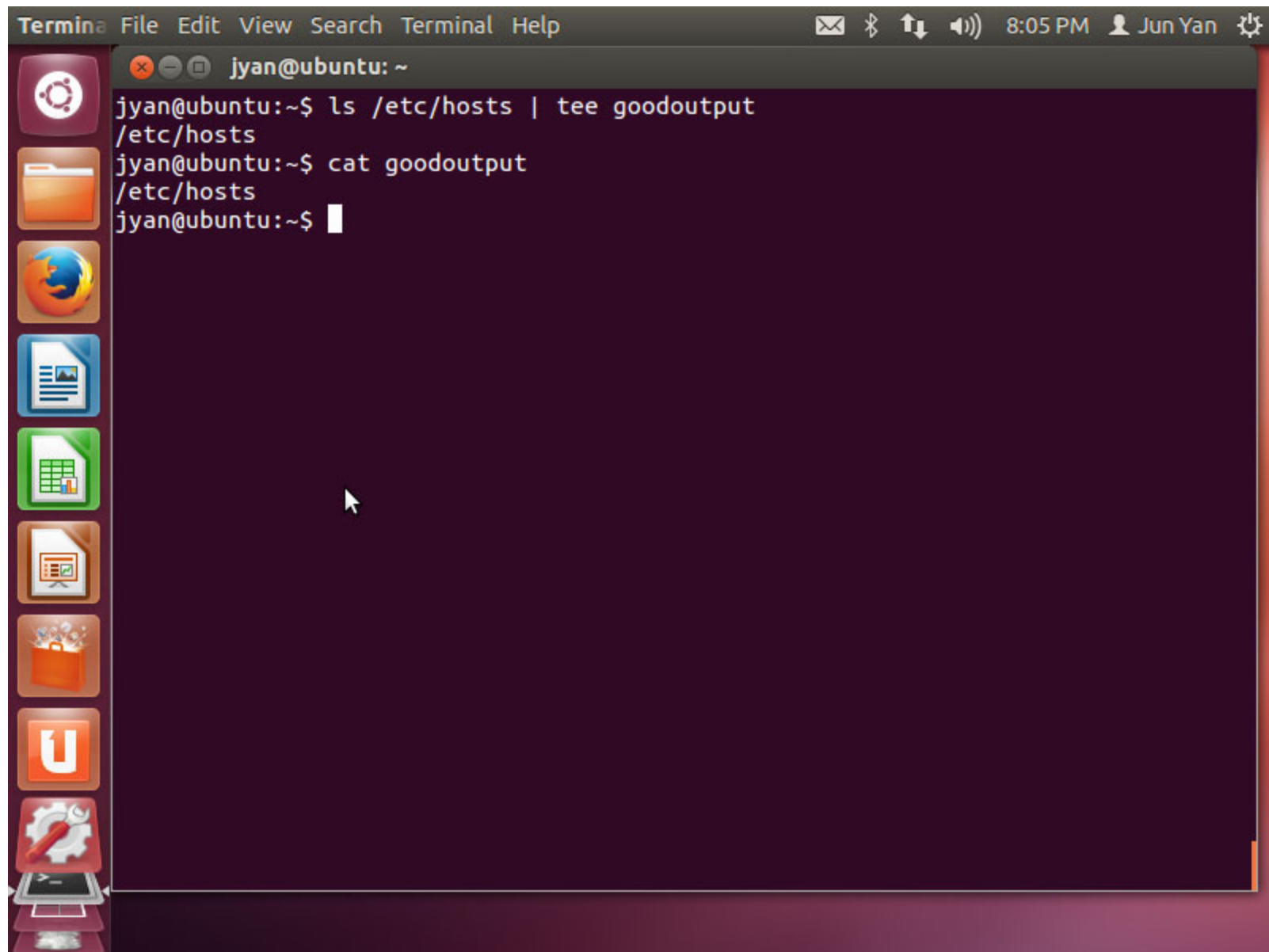
- List /etc page by page
- Replace all lowercase a with uppercase A in /etc/passwd and display the content in the terminal screen page by page
- Replace all lowercase a with uppercase A in /etc/passwd, sort the contents by the first character on each line, and then save the results to /home/jyan/results



Terminal

jyan@ubuntu: ~

```
jyan@ubuntu:~$ ls -l /etc | more
total 1136
drwxr-xr-x 3 root root 4096 Aug 7 2014 acpi
-rw-r--r-- 1 root root 2981 Aug 7 2014 adduser.conf
-rw-r--r-- 1 root root 10 Mar 15 2017 adjtime
drwxr-xr-x 2 root root 4096 Apr 3 2017 alternatives
-rw-r--r-- 1 root root 395 Jun 20 2010 anacrontab
-rw-r--r-- 1 root root 112 Jun 22 2007 apg.conf
drwxr-xr-x 6 root root 4096 Aug 7 2014 apm
drwxr-xr-x 3 root root 4096 Apr 3 2017 apparmor
drwxr-xr-x 8 root root 4096 Apr 3 2017 apparmor.d
drwxr-xr-x 5 root root 4096 Apr 3 2017 appport
drwxr-xr-x 6 root root 4096 Apr 3 2017 apt
-rw-r----- 1 root daemon 144 Oct 25 2011 at.deny
drwxr-xr-x 2 root root 4096 Aug 7 2014 at-spi2
drwxr-xr-x 3 root root 4096 Apr 3 2017 avahi
-rw-r--r-- 1 root root 2076 Apr 3 2012 bash.bashrc
-rw-r--r-- 1 root root 58753 May 10 2013 bash_completion
drwxr-xr-x 3 root root 4096 Apr 3 2017 bash_completion.d
-rw-r--r-- 1 root root 356 Apr 19 2012 bindresvport.blacklist
-rw-r--r-- 1 root root 321 Mar 29 2012 blkid.conf
lrwxrwxrwx 1 root root 15 Mar 15 2017 blkid.tab -> /dev/.blkid.tab
drwxr-xr-x 2 root root 4096 Aug 7 2014 bluetooth
-rw-r--r-- 1 root root 33 Aug 7 2014 brlapi.key
--More--
```



The image shows a terminal window titled "Terminal" with a menu bar containing "File", "Edit", "View", "Search", "Terminal", and "Help". The window has a dark purple background. On the left side, there is a vertical dock with several application icons: a red circle with a white dot, a folder, a globe, a document with a bar chart, a spreadsheet, a presentation, a shopping bag, a red 'U' logo, and a gear with a wrench. The terminal prompt is "jyan@ubuntu: ~". The user has entered the command "ls /etc/hosts | tee goodoutput", which has been executed, and the output "/etc/hosts" is displayed. The user then enters "cat goodoutput", and the output "/etc/hosts" is displayed again. The prompt "jyan@ubuntu:~\$" is shown at the end of the line.

```
Terminal File Edit View Search Terminal Help
jyan@ubuntu: ~
jyan@ubuntu:~$ ls /etc/hosts | tee goodoutput
/etc/hosts
jyan@ubuntu:~$ cat goodoutput
/etc/hosts
jyan@ubuntu:~$
```

Terminal

6:30 PM Jun Yan

```
jyan@ubuntu: ~  
jyan@ubuntu:~$ cat file1  
dsdfdf  
dfdf  
ddd  
ddddddddddddd  
ddgfrglv  
jyan@ubuntu:~$ tr d D <file1 | sort | nl >file2  
jyan@ubuntu:~$ cat file2  
1 DDD  
2 DDDDDDDDDDDDD  
3 DDgfrglv  
4 DfDf  
5 DsDfDf  
jyan@ubuntu:~$
```

Shell variables

- Environment variables are typically set by the system and contain information that the system and programs access regularly
- User-defined variables are created by users
- There are other special variables that are useful when executing commands and creating new files and directories



Environment variables

- Use the *set* command or *env* command to see a list of environment variables and their values
 - HOME: the absolute pathname to the user's home directory
 - PWD: the present working directory in the directory tree
 - PATH: a list of directories to search for executable programs
 - SHELL: the absolute pathname of the current shell
 - LOGNAME: the username of the current user used when logging into the shell
 - TZ: the time zone for the system



User-defined variables

- Variable names are prefixed with a dollar sign when their values are referenced
 - `echo $MYVAR`
- You can surround the variable name with curly braces
 - `echo "I am the ${REV}th in the competition."`
- Variables created in the current shell are only available to the current shell, not to the subshells, unless you export them



Setting variables

- To change the value of a variable, specify the variable name immediately followed by an equal sign (=) and the new value
- In some distributions, you must use the *setenv* command followed by the variable name and the new value.
- Variable names can contain alphanumeric characters, dash (-) or underscore (_)
- They must not start with a number
- They are typically capitalised to follow convention



```
Terminal
jyan@ubuntu: ~
HOME=/home/jyan
SHLVL=2
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
LOGNAME=jyan
XDG_DATA_DIRS=/usr/share/ubuntu:/usr/share/gnome:/usr/local/share:/usr/share/
DBUS_SESSION_BUS_ADDRESS=unix:abstract=/tmp/dbus-0o9WwpB09W,guid=78dfa5c96e825bf
efbff9fdd00000028
LESSOPEN=| /usr/bin/lesspipe %s
DISPLAY=:0
XDG_CURRENT_DESKTOP=Unity
LESSCLOSE=/usr/bin/lesspipe %s %s
XAUTHORITY=/home/jyan/.Xauthority
COLORTERM=gnome-terminal
_=/usr/bin/env
OLDPWD=/usr/bin
jyan@ubuntu:~$ echo $MYVAR
this is my variable
jyan@ubuntu:~$ env | grep MYVAR
jyan@ubuntu:~$ set | grep MYVAR
MYVAR='this is my variable'
jyan@ubuntu:~$ export MYVAR
jyan@ubuntu:~$ env | grep MYVAR
MYVAR=this is my variable
jyan@ubuntu:~$
```

Other variables

- Other variables are not displayed by `set` or `env` commands, and perform specialised functions
 - `UMASK` is a special variable (discussed in the next lecture)
- Command aliases are shortcuts to commands stored in special variables that can be created and viewed by the `alias` command
 - `alias dw="date;who"`



```
Terminal
jyan@ubuntu: ~

jyan@ubuntu:~$ alias
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo terminal || echo error)" "$(history|tail -n1|sed -e '\''s/^\s*[0-9]\+\s*//;s/[\;&]\s*alert$//'\''
)'"'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -alF'
alias ls='ls --color=auto'
jyan@ubuntu:~$ alias dw="date;who"
jyan@ubuntu:~$ dw
Sun Mar 18 18:56:14 PDT 2018
jyan      tty7          2018-03-18 17:05
jyan      pts/0          2018-03-18 17:07 (:0)
jyan@ubuntu:~$ alias
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo terminal || echo error)" "$(history|tail -n1|sed -e '\''s/^\s*[0-9]\+\s*//;s/[\;&]\s*alert$//'\''
)'"'
alias dw='date;who'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -alF'
alias ls='ls --color=auto'
jyan@ubuntu:~$
```

Bash scripting

- Bash scripts automate things you'd otherwise be typing on the command line
 - The first line is known as the “shebang” statement and declares the text file to be a script for interpretation by bash:
 - `#!/bin/bash`
 - Comments start with a hash mark (#)
 - Invoke the shell as an interpreter or turn on the execution bit of the script to run the script
 - No need to set the file's extension but some people use `.sh`



Terminal

9:09 PM Jun Yan

```
jyan@ubuntu: ~  
jyan@ubuntu:~$ vi firstscript  
jyan@ubuntu:~$ vi firstscript  
jyan@ubuntu:~$ cat firstscript  
#!/bin/bash  
date  
#this is a comment  
echo "hello world"  
jyan@ubuntu:~$ bash firstscript  
Thu Mar  1 21:08:57 PST 2018  
hello world  
jyan@ubuntu:~$ source firstscript  
Thu Mar  1 21:09:06 PST 2018  
hello world  
jyan@ubuntu:~$
```


Command-line arguments and functions

- Command-line arguments to a script become variables whose names are numbers
 - \$1 is the first command-line argument, and so on
 - \$0 is the name by which the script was invoked
 - \$# contains the number of command-line arguments that were supplied
 - \$* contains all the arguments at once



Terminal

9:31 PM Jun Yan

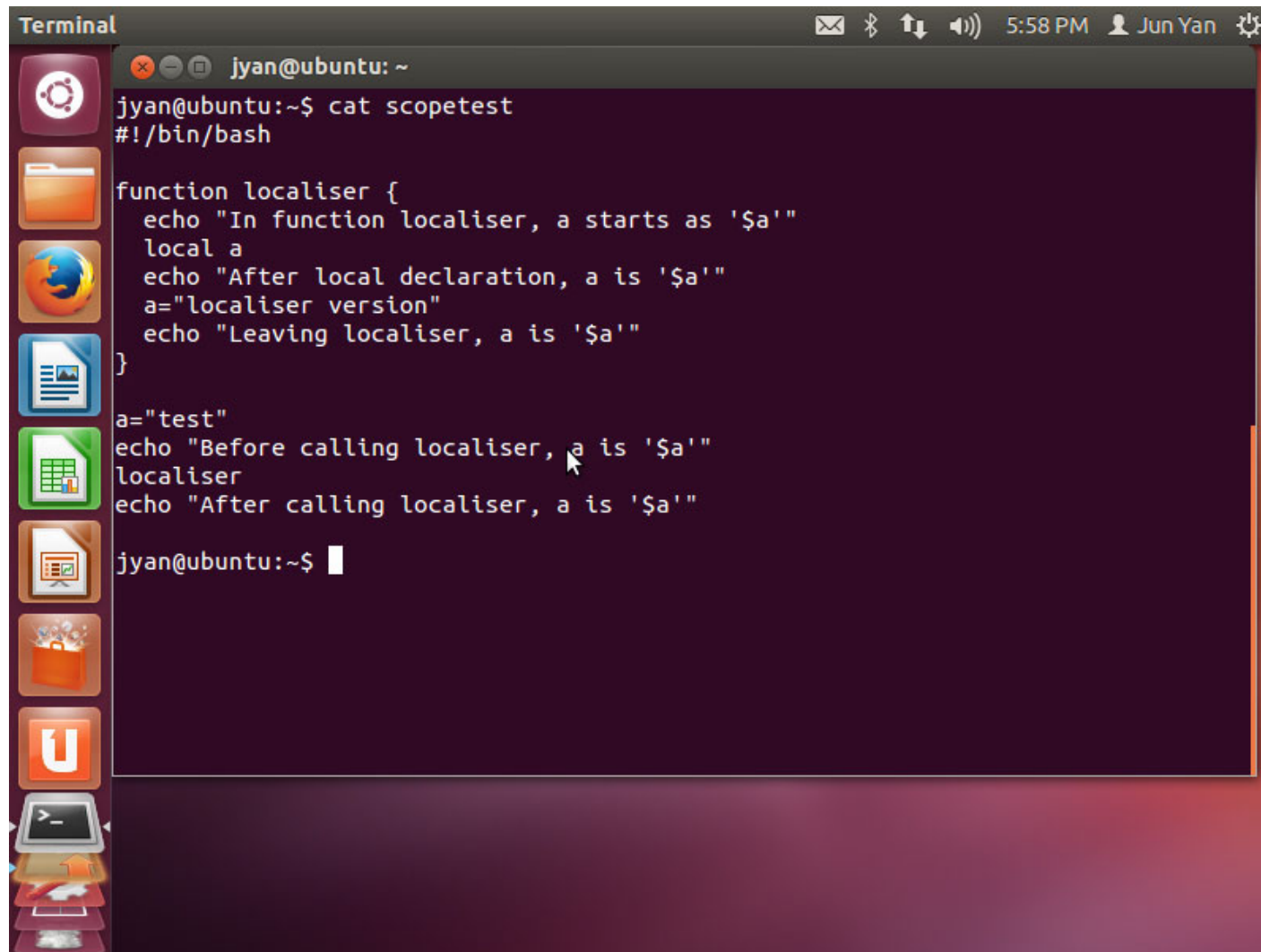
```
jyan@ubuntu: ~  
jyan@ubuntu:~$ cat secondscript  
#!/bin/bash  
function show_usage {  
    echo "Usage: $0 source_dir dest_dir"  
    exit 1  
}  
  
if [ $# -ne 2 ]; then  
    show_usage  
else  
    printf "Source directory is $1\n"  
    printf "Destination directory is $2\n"  
fi  
jyan@ubuntu:~$ bash secondscript  
Usage: secondscript source_dir dest_dir  
jyan@ubuntu:~$ bash secondscript dir1 dir2  
Source directory is dir1  
Destination directory is dir2  
jyan@ubuntu:~$
```

Variable scope

- Variables are global within a script
- Functions can create their own local variables with a local declaration
 - `local variablename`



What is the output

A terminal window titled "Terminal" with a dark purple background. The window shows a shell script being executed. The script defines a function named "localiser" which prints the value of a variable 'a' at different stages. It then sets 'a' to "test", calls the "localiser" function, and prints 'a' again. The terminal output shows the script's execution steps and the resulting values of 'a'.

```
Terminal
jyan@ubuntu: ~
jyan@ubuntu:~$ cat scopetest
#!/bin/bash

function localiser {
    echo "In function localiser, a starts as '$a'"
    local a
    echo "After local declaration, a is '$a'"
    a="localiser version"
    echo "Leaving localiser, a is '$a'"
}

a="test"
echo "Before calling localiser, a is '$a'"
localiser
echo "After calling localiser, a is '$a'"

jyan@ubuntu:~$
```



Control flow

- if-then and if-then-else
- The terminator for an if statement is `fi`
- The `elif` keyword to mean “else if”
- See the example in the next slide. Both the peculiar `[]` syntax for comparisons and the integer comparison operators (e.g., `-eq`) are inherited from the original Bourne Shell’s channelling of `/bin/test`.
- The brackets actually invoke `test` and are not a syntactic requirement of the if statement



Terminal

4:37 PM Jun Yan

```
jyan@ubuntu: ~  
jyan@ubuntu:~$ cat thirdscript  
#!/bin/bash  
function show_usage {  
    echo "Usage: $0 source_dir dest_dir"  
    exit 1  
}  
  
if [ $# -ne 2 ]; then  
    show_usage  
elif [ -d $1 ] && [ -d $2 ]; then  
    printf "Source directory is $1\n"  
    printf "Destination directory is $2\n"  
else  
    show_usage  
fi  
jyan@ubuntu:~$ bash thirdscript aaa bbb  
Usage: thirdscript source_dir dest_dir  
jyan@ubuntu:~$
```

Elementary bash comparison operators

String	Numeric	True if
<code>x = y</code>	<code>x -eq y</code>	x is equal to y
<code>x != y</code>	<code>x -ne y</code>	x is not equal to y
<code>x < y</code>	<code>x -lt y</code>	x is less than y
<code>x <= y</code>	<code>x -le y</code>	x is less than or equal to y
<code>x > y</code>	<code>x -gt y</code>	x is greater than y
<code>x >= y</code>	<code>x -ge y</code>	x is greater than or equal to y
<code>-n x</code>	-	x is not null
<code>-z x</code>	-	x is null



bash file evaluation operators

Operator	True if
-d file	file exists and is a directory
-e file	file exists
-f file	file exists and is a regular file
-r file	You have read permission on the file
-s file	file exists and is not empty
-w file	You have write permission on the file
file1 -nt file2	file1 is newer than file2
file1 -ot file2	file1 is older than file2



Loops

- bash's for...in construct takes some action for a group of values
- Any whitespace-separated list of things, including the contents of a variable, works as a target of for...in
- for loop

```
for (( i=0; i < $count; i++)); do
    ...
done
```
- while loop
- See examples in the next a few slides



```
Terminal
jyan@ubuntu: ~
badoutput Downloads fourthscript Public Videos
Desktop examples.desktop goodoutput secondscript
dir1 file1 Music Templates
Documents firstscript Pictures thirdscript
jyan@ubuntu:~$ clear
jyan@ubuntu:~$ cat fourthscript
#!/bin/bash
suffix=BACKUP--`date +%Y%m%d`
for script in *script; do
    newname="$script.$suffix"
    echo "Copying $script to $newname..."
    cp $script $newname
done
jyan@ubuntu:~$ bash fourthscript
Copying firstscript to firstscript.BACKUP--20180304...
Copying fourthscript to fourthscript.BACKUP--20180304...
Copying secondscript to secondscript.BACKUP--20180304...
Copying thirdscript to thirdscript.BACKUP--20180304...
jyan@ubuntu:~$ ls
badoutput firstscript Public
Desktop firstscript.BACKUP--20180304 secondscript
```

Terminal

5:14 PM Jun Yan

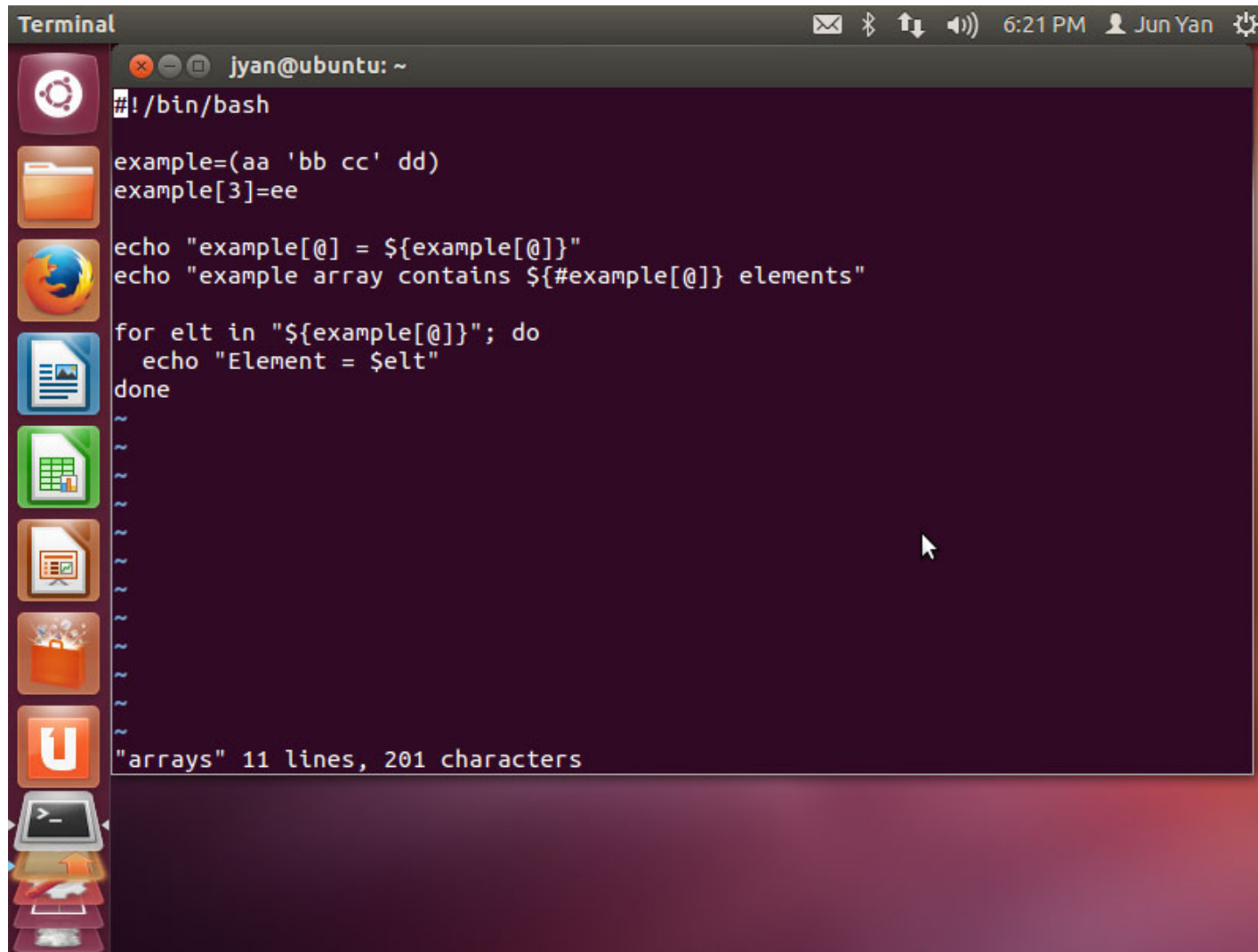
```
jyan@ubuntu: ~  
jyan@ubuntu:~$ cat fifthsript  
#!/bin/bash  
  
exec 0<$1  
while read line; do  
    echo "$line"  
done  
jyan@ubuntu:~$ bash fifthsript fifthsript  
#!/bin/bash  
  
exec 0<$1  
while read line; do  
    echo "$line"  
done  
jyan@ubuntu:~$
```

Arrays

- Arrays are not often used in bash.
- Literal arrays are delimited by parentheses, and the elements are separated by whitespace
 - `example=(aa bb cc dd)`
- Use `${array_name[subscript]}` to access individual elements
 - `echo ${example[1]}`
- The subscripts `*` and `@` refer to the array as a whole
 - `${example[@]}`
 - `${#example[@]}` yields the number of elements in the array



What is the output?



A terminal window titled "Terminal" with a dark purple background. The window shows a bash shell session for user "jyan" on an "ubuntu" machine. The prompt is "#!/bin/bash". The script defines an array "example" with elements "aa", "bb", "cc", and "dd", then appends "ee" to the third element. It then prints the array's contents and iterates over each element, printing "Element = " followed by the element value. The script ends with a status message: "arrays" 11 lines, 201 characters. The terminal window has a sidebar with icons for various applications and a top bar with system status icons and the time "6:21 PM".

```
#!/bin/bash

example=(aa 'bb cc' dd)
example[3]=ee

echo "example[@] = ${example[@]}"
echo "example array contains ${#example[@]} elements"

for elt in "${example[@]}"; do
    echo "Element = $elt"
done

"arrays" 11 lines, 201 characters
```

Regular expressions

- Regular expressions enable you to specify a certain pattern of text within a text document.
- Similar to wildcard metacharacters but interpreted by a text tool program.
- More regular expression metacharacters are available than wildcard metacharacters.



Symbol	Description
.	Matches any one character
*	Matches zero or more occurrences of the previous character.
?	Matches zero or one occurrence of the previous character.
+	Matches one or more occurrences of the previous character.
[chars]	Matches any character from a given set, e.g., [a-z]
[^chars]	Matches any character not in a given set, e.g., [^a-z]
()	Matches either the element to its left or to its right.



Symbol	Description
<code>^</code>	Matches the beginning of a line.
<code>\$</code>	Matches the end of a line.
<code>\w</code>	Matches any word character. [a-zA-Z0-9_]
<code>\s</code>	Matches any whitespace character.
<code>\d</code>	Matches any digit. [0-9]
<code>{n}</code>	Matches exactly n instances of the preceding element.
<code>{min,}</code>	Matches at least min instances of the preceding element.
<code>{min,max}</code>	Matches any number of instances of the preceding element from min to max.



Examples

- `^\d{4}$`
- `Letter[^1238]`
- `(mother|father)`
- `M[ou]’?am+[ae]r ([Aeae][[-
])?[GKQ]h?[aeu]+([dhz]|[dhz]{1,2})af[iy]`



The grep command

- Global Regular Expression Print searches for information using regular expressions
- It displays the lines of text that match extended regular expressions.
 - `grep "hello" file1`
 - `grep -v "hello" file1`
 - `grep -i "hello" file1`
 - `grep "^I " file1`



Perl programming

- Perl is a scripting language that offers vastly more power than bash
- It is a better choice for systems administration than traditional programming language (the other choice is Python)
- Perl can do more in fewer lines of code, with less painful debugging
- Perl's catch phrase is that “there's more than one way to do it”



Perl programming

- Perl statements are separated by semicolons
- Comments start with a hash mark (#)
- Blocks of statements are enclosed in curly braces
- Lines in a perl script are not shell commands; they are perl codes
- The first line is known as the “shebang” statement and declares the text file to be a script for interpretation by perl:
 - `#!/usr/bin/perl`



Variables and arrays

- Perl has three fundamental data types:
 - Scalars: scalar variables start with \$
 - \$a=“this is an example”;
 - \$count=0;
 - Arrays: array variables start with @
 - @items=(“socks”, “shoes”, “shorts”);
 - Array subscripting begins at zero, the index of the highest element is \$#items
 - The array @ARGV contains the script’s command-line arguments
 - Hashes (aka associative arrays): hash variables start with %



What is the output?

[illegible]

Hashes

- A hash represents a set of key/value pairs.
- In other words, a hash is an array whose subscripts are arbitrary scalar values
- Subscripting is indicated with curly braces rather than square brackets
 - `$myhash{'ron'}`



What is the output?

[illegible]

Elementary Perl comparison operators

String	Numeric	True if
<code>x eq y</code>	<code>x = y</code>	x is equal to y
<code>x ne y</code>	<code>x != y</code>	x is not equal to y
<code>x lt y</code>	<code>x < y</code>	x is less than y
<code>x le y</code>	<code>x <= y</code>	x is less than or equal to y
<code>x gt y</code>	<code>x > y</code>	x is greater than y
<code>x ge y</code>	<code>x >= y</code>	x is greater than or equal to y

You get all the file-testing operators shown in Slide 48 except for the `-nt` and `-ot` operators



Control flow

- Control flow in Perl is very similar to that in bash
- if construct has no then keyword or terminating word



Input and output

- When you open a file for reading or writing, you define a filehandle to identify the channel
 - `open($fh, '<', $inputfilename) || die "Couldn't open file.";`
 - `open($fh, '>', $outputfilename) || die "Couldn't open file.";`



Questions?



UNIVERSITY
OF WOLLONGONG
AUSTRALIA